

COMIC CRAFT

AI Comic Story Creator Using Gemini Models

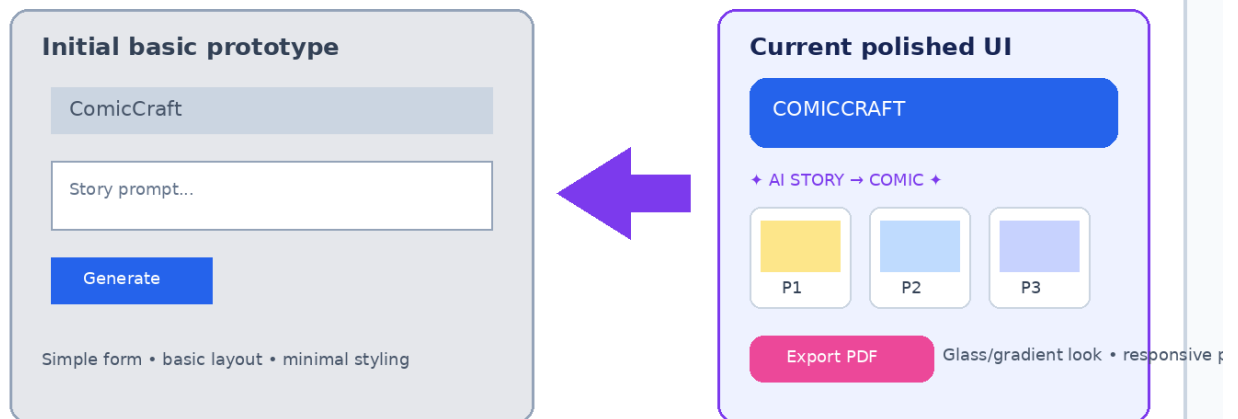
PROJECT DOCUMENTATION & DEMONSTRATION REPORT

Government Arts College for Women
Salem – 08
(PER 104)

Project Theme

From a simple story prompt to a structured, illustrated, multi-panel comic with AI-assisted storytelling, character consistency, dialogue presentation and PDF export.

ComicCraft UI Evolution



Prepared for project demonstration and academic documentation

Contents

- 1. Project Overview
- 2. Abstract
- 3. Introduction
- 4. Problem Statement
- 5. Objectives
- 6. Existing System
- 7. Proposed System
- 8. Evolution of the User Interface
- 9. System Architecture
- 10. Technologies and Tools Used
- 11. Gemini Model Integration
- 12. Character Consistency Mechanism
- 13. Comic Generation Workflow
- 14. User Interface and Current Outlook
- 15. Sample Demonstration
- 16. Speech Bubble and Dialogue Handling
- 17. PDF Export
- 18. Error Handling and Reliability
- 19. Testing and Validation
- 20. Advantages
- 21. Limitations
- 22. Future Enhancements
- 23. Applications
- 24. Conclusion
- 25. Demonstration Guide
- 26. References

1. Project Overview

ComicCraft is an AI-assisted comic story creation application designed to transform a user's story idea into a structured five-panel comic. The project combines a web interface, a Python/FastAPI backend, Google Gemini models for intelligent text generation, image-generation services, dialogue presentation and PDF export.

The central idea is to reduce the number of manual steps involved in comic creation. Instead of separately writing a story, planning panels, defining characters and preparing image prompts, the user supplies a story concept and ComicCraft coordinates these stages through a single workflow.

Core value

ComicCraft connects natural-language storytelling with a structured comic-production pipeline while giving special attention to character identity and panel consistency.

2. Abstract

ComicCraft – AI Comic Story Creator Using Gemini Models is a web-based artificial-intelligence project that converts a natural-language story prompt into a multi-panel comic. The application uses Python and FastAPI for

backend orchestration, Pydantic models for structured data validation, and the Google GenAI client for Gemini-based story and prompt generation.

The generation pipeline first creates a five-panel outline, then creates a permanent character identity profile containing species/type, appearance, clothing or accessories, distinctive features and an identity description. That identity is reused when generating panel-specific image prompts. The resulting artwork is presented through a comic preview interface together with dialogue and other panel information, and the completed comic can be exported as a PDF.

The project demonstrates how generative AI can be combined with conventional web-development techniques to build an end-to-end creative application. The system also includes validation and retry handling for temporary AI-service failures.

3. Introduction

Comic creation traditionally requires multiple creative and technical tasks: story writing, character design, panel planning, illustration, dialogue placement and final page layout. These tasks can be time-consuming when performed independently. Generative AI makes it possible to automate several of these stages while still allowing the user to start with a simple story idea.

ComicCraft was developed as a practical demonstration of this concept. The application treats Gemini as an intelligent story-planning component and combines its structured outputs with an image-generation and presentation pipeline.

- Natural-language input is used as the starting point.
- Gemini generates structured story information rather than only free-form prose.
- A reusable character profile is created before panel image prompts are produced.
- Each panel receives a dedicated image prompt with composition and consistency instructions.
- The generated panels are displayed in a comic-oriented interface.
- Dialogue and speech-bubble presentation are incorporated into the visual preview.
- The final comic can be exported for offline use and demonstration.

4. Problem Statement

A user who wants to create a short comic normally has to convert an idea into a complete sequence of scenes and then prepare artwork and dialogue for each scene. A major challenge is maintaining the identity of the main character when separate images are generated for separate panels.

- Manual conversion of a story idea into panel-by-panel scenes.
- Difficulty maintaining the same character appearance across independently generated images.
- Time required to write detailed image prompts for every panel.
- Need to leave appropriate space for speech bubbles and dialogue.
- Need to assemble generated artwork into a coherent comic layout.
- Need for a convenient export format such as PDF.
- Dependence on external AI services can introduce temporary availability or timeout errors.

5. Objectives

1. Develop a simple web-based interface for comic creation.
2. Generate a structured five-panel story outline from a user prompt.
3. Generate a reusable character identity profile.

- 4. Use the identity profile in every panel's image-generation prompt.
- 5. Create detailed image prompts automatically.
- 6. Generate and display comic artwork.
- 7. Present dialogue through comic-style speech bubbles in the preview.
- 8. Provide PDF export for the completed comic.
- 9. Validate AI-generated JSON before using it in the application.
- 10. Demonstrate a practical end-to-end generative-AI workflow.

6. Existing System

The initial concept of ComicCraft began as a basic prototype webpage. The early interface focused on collecting the user's story details and triggering generation. It provided the functional foundation but had a relatively plain appearance and limited visual hierarchy.

In a traditional/manual workflow, the user would need to move between separate tools for writing, image creation, dialogue placement and final layout. The prototype demonstrated the core idea but did not yet provide the polished comic-oriented presentation that the current version targets.

Initial prototype characteristics

Basic form-oriented webpage • minimal visual styling • direct generation controls • limited preview structure • functional rather than presentation-focused design.

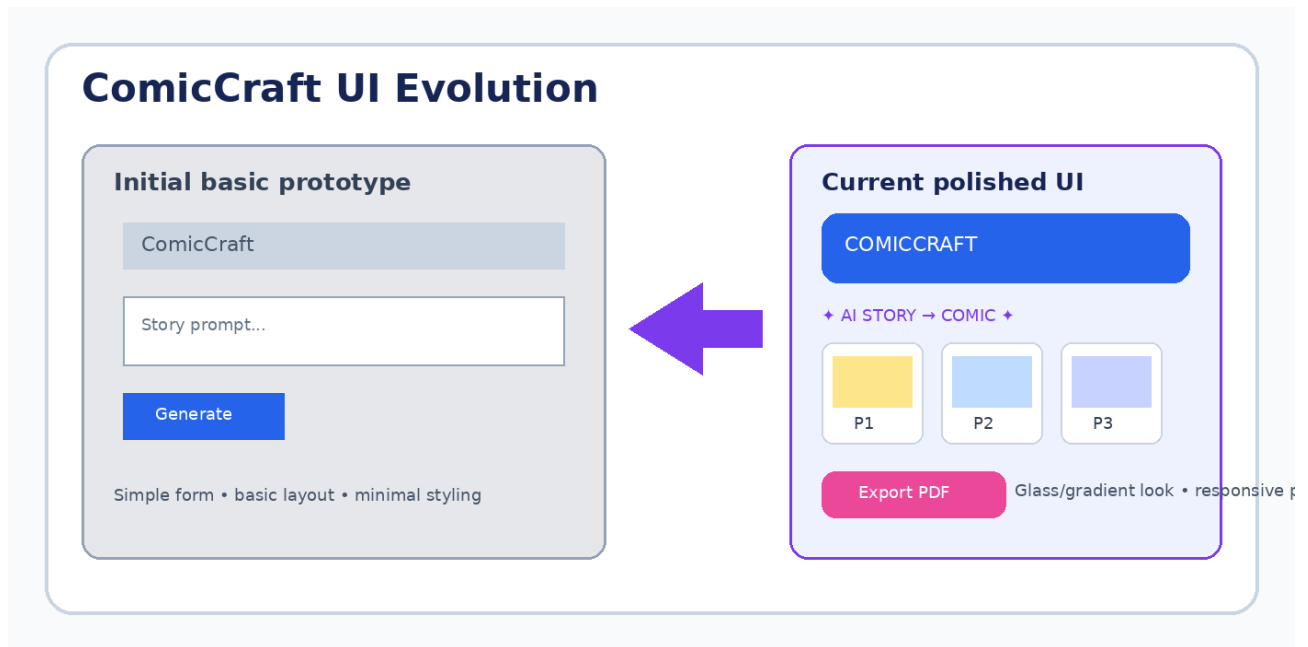
7. Proposed System

The current ComicCraft design integrates story processing, character consistency, image prompting, comic preview and export into a single workflow. The backend coordinates the AI stages while the frontend focuses on input, feedback and visual presentation.

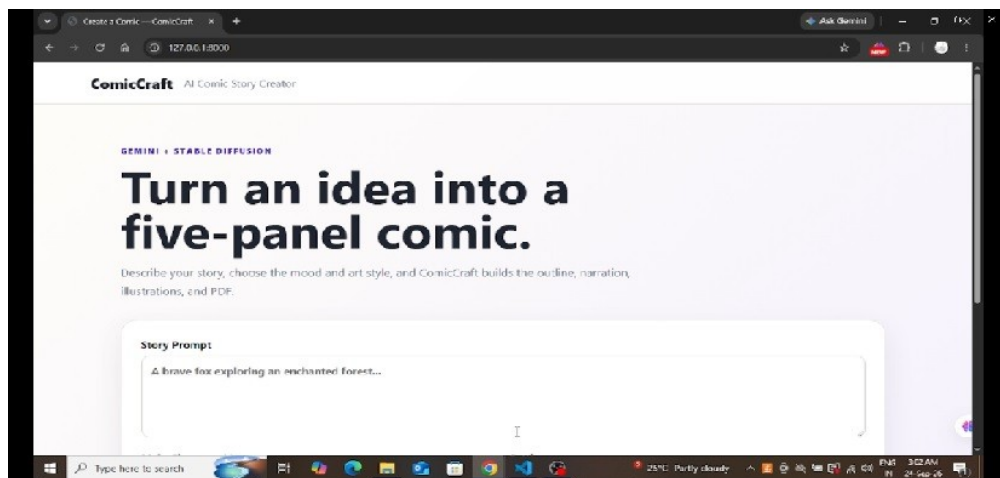
Stage	Function	Output
1. Input	Collect story prompt and settings	PromptRequest
2. Story planning	Gemini creates exactly five panels	Panel outline JSON
3. Character identity	Gemini defines stable visual identity	CharacterProfile
4. Prompt generation	Create detailed prompt for each panel	Image prompt
5. Image generation	Generate panel artwork	PNG panel
6. Preview	Arrange panels and dialogue	Comic preview
7. Export	Create printable document	PDF

8. Evolution of the User Interface

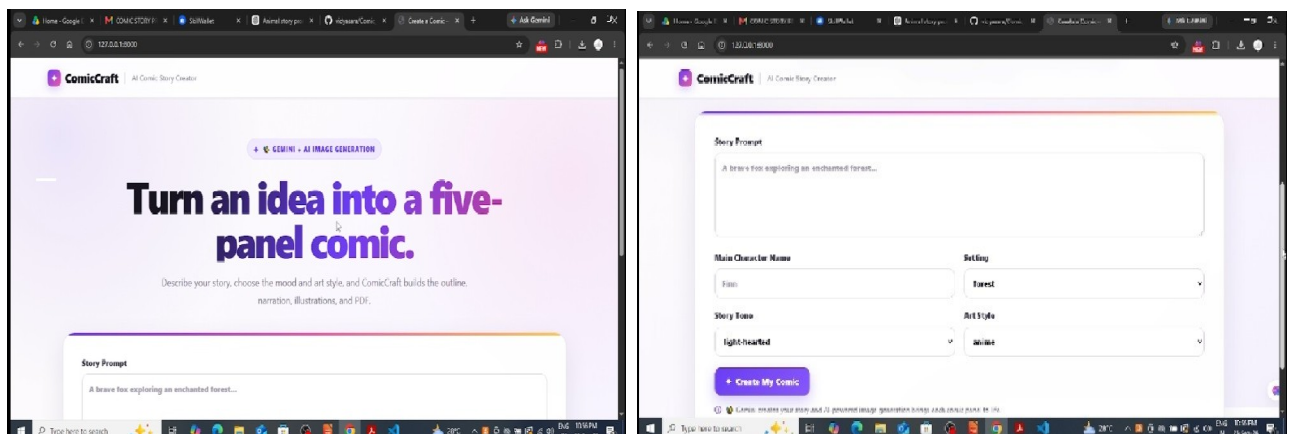
One of the major improvements in the project was the transition from the original basic prototype webpage to the current polished ComicCraft presentation. The change was not only cosmetic; the interface was reorganized around the way a user actually experiences a comic-generation workflow.



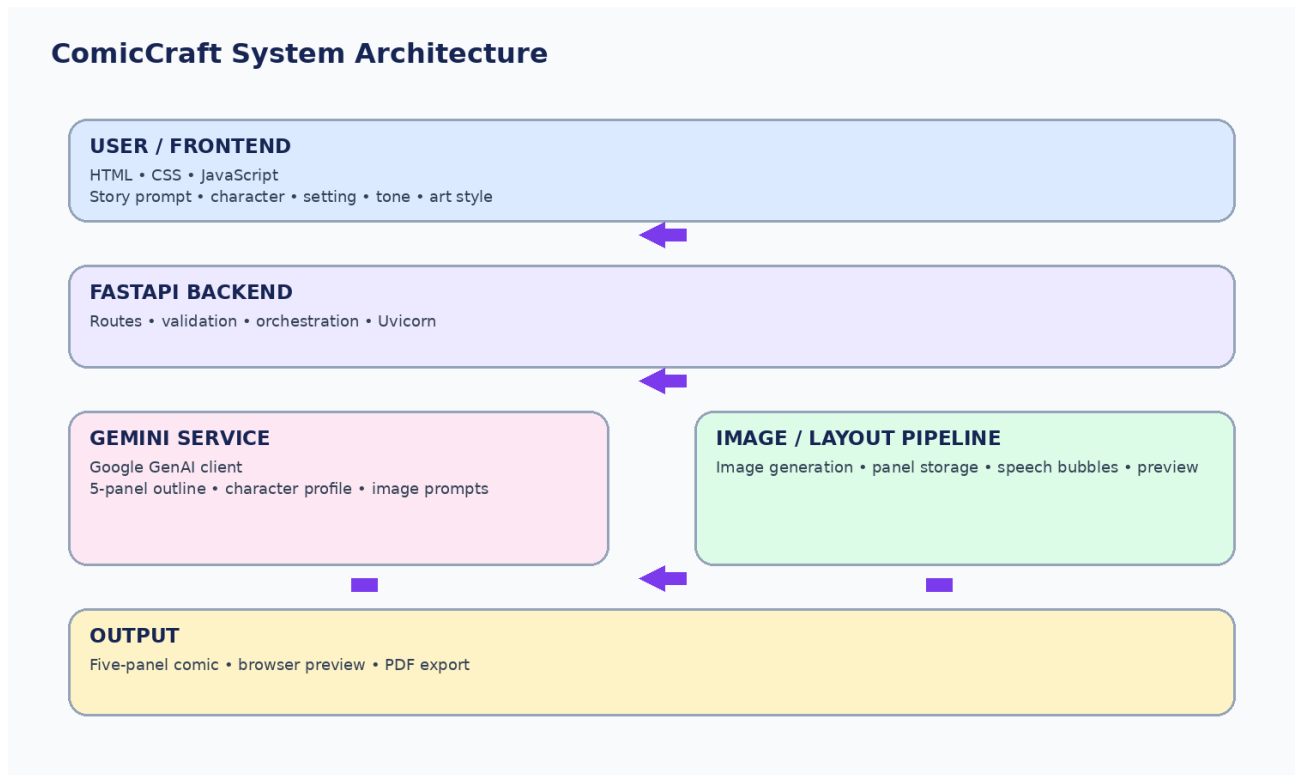
Initial prototype: a simple form-based page intended mainly to prove that story input and generation could be triggered.



Current outlook: a more polished gradient/glass-inspired homepage, stronger visual hierarchy, responsive two-column preview, panel cards, lazy-loaded images, comic-style speech bubbles over artwork and clearer presentation of generated content.



9. System Architecture



The architecture separates the user-facing interface, backend orchestration, Gemini-based text generation, image/layout processing and final output. This separation makes the project easier to test and extend.

- Frontend: HTML templates, CSS styling and JavaScript interaction.
- Backend: FastAPI routes and Python service modules.
- AI layer: Google GenAI client and Gemini model configuration.
- Data layer: Pydantic models and JSON validation.
- Image layer: generated panel images and static asset serving.
- Presentation/export layer: comic preview and PDF export.

10. Technologies and Tools Used

Technology / Tool	Role in ComicCraft
Python	Primary programming language for backend logic and services.
FastAPI	Web framework used for application routes and request handling.
Uvicorn	Development/application server used to run the FastAPI application.
Google GenAI client	Connects the application to Gemini models.
Gemini 2.5 Flash	Text-generation model configured for story and prompt generation.
Pydantic	Validation and structured data models such as PromptRequest, CharacterProfile and PanelOutline.
JSON	Structured interchange format for Gemini outputs such as panel outlines and character profiles.

PyTorch	AI/image-processing support within the project environment.
Diffusers	Image-generation pipeline support in the project.
HTML/CSS/JavaScript	Frontend structure, styling and browser interaction.
PDF export module	Converts the generated comic content into a shareable PDF.

11. Gemini Model Integration

The application uses the Google GenAI Python client to communicate with Gemini. The client is created from the API key stored in the application's settings. The configured Gemini model is passed as a string to generate_content(), while the story prompt is passed as the request contents.

```
response = client.models.generate_content(  
    model=model,  
    contents=prompt,  
)  
  
return response.text.strip()
```

The project uses Gemini in several distinct stages rather than asking for the entire comic in one response. This staged approach makes it possible to validate structured information and carry important character details into later prompts.

11.1 Structured Gemini Output

The story outline is requested as JSON containing panel number, title and scene description. The character profile is also requested as JSON and validated using the CharacterProfile Pydantic model. This reduces the risk of passing malformed AI output into later stages.

11.2 Error Handling

The Gemini text-generation helper retries failed requests. During testing, temporary 503 UNAVAILABLE and 504 DEADLINE_EXCEEDED responses were observed. A successful Fox-story generation was completed, while a later attempt encountered repeated 503 high-demand responses. These observations demonstrate that external AI-service availability can vary independently of the application's story logic.

12. Character Consistency Mechanism

Character consistency is one of the core technical ideas of ComicCraft. Instead of independently describing the character in every panel, the system first creates a permanent visual identity profile.

Profile field	Purpose
Name	Identifies the main character.
Species / Type	Prevents accidental conversion into another character type.
Appearance	Stores stable physical characteristics.
Clothing / Accessories	Defines recurring visual elements.
Distinctive Features	Provides recognizable identifying details.
Identity Description	Compact detailed description reused in image prompts.

The profile is then inserted into every panel image prompt with explicit instructions that the character identity is locked. The prompt also prohibits changing the species/type or replacing the character with another character.

```
PERMANENT CHARACTER IDENTITY:
```

```
Name: ...
Species / Type: ...
Appearance: ...
Clothing / Accessories: ...
Distinctive Features: ...
Identity Description: ...

THIS CHARACTER IDENTITY IS LOCKED.
```

13. Comic Generation Workflow

11. User enters the story idea and selects character, setting, tone and art style.
12. FastAPI receives the request and converts it into a validated PromptRequest.
13. Gemini creates exactly five panel descriptions.
14. Gemini creates a permanent CharacterProfile.
15. For each panel, Gemini creates a detailed image-generation prompt.
16. The image prompt includes character identity, action, environment, lighting, composition and camera direction.
17. Composition instructions reserve clean visual space for speech bubbles.
18. The image-generation service produces panel artwork.
19. The generated panels are stored and served through the application's static panel route.
20. The preview page assembles the panels, dialogue and comic information.
21. The export route produces the final PDF.

14. User Interface and Current Outlook

The current interface is designed to feel more like a dedicated creative application than a basic prototype form. The homepage uses a polished visual language, while the preview focuses attention on the generated comic.

- Gradient/glass-inspired visual treatment on the homepage.
- Responsive two-column preview structure.
- Panel-card presentation for individual comic scenes.
- Lazy-loaded panel images to improve browser behavior.
- Speech bubbles positioned over comic artwork in the preview.
- Speaker-name handling so dialogue can be presented cleanly.
- Clear export action for PDF creation.
- Consistent typography and visual hierarchy.

15. Sample Demonstration

A useful demonstration story is “The Brave Little Fox”. The story follows Finn, a young fox who discovers a bird that has fallen from its nest. Finn helps the bird return safely, receives a warm response from the birds and learns that a small act of kindness can make a big difference.

Input field	Demonstration value
Story prompt	Tests natural-language story understanding.
Character: Finn	Tests character identity generation.

Setting: Forest	Tests environment consistency.
Tone: Warm and funny	Tests stylistic control.
Art style: Colorful children's comic	Tests visual prompt control.

16. Speech Bubble and Dialogue Handling

The current preview layer presents dialogue in comic-style speech bubbles over the generated artwork. The layout logic preserves Gemini-provided dialogue information and prepares it for display with the panel content.

The image prompts explicitly instruct the image generator not to render dialogue, speech bubbles, captions, letters or written words inside the artwork. This separation is important because it allows the application layer to control readable dialogue presentation instead of depending on an image model to render text accurately.

Design principle

Artwork generation and dialogue rendering are treated as separate responsibilities: the image model creates the scene, while ComicCraft controls readable comic dialogue in the presentation layer.

17. PDF Export

ComicCraft provides an export route that turns the generated comic into a PDF suitable for saving, sharing and demonstration. The export stage is intentionally separated from the browser preview so that the user can inspect the comic before creating the final document.

The PDF export is one of the final user-visible stages of the pipeline. During project testing, the Fox story successfully reached final output, while temporary Gemini service failures were also observed during separate attempts.

18. Error Handling and Reliability

Observed / anticipated condition	Application response
Missing Gemini API key	Raises a clear configuration <code>RuntimeError</code> .
Invalid Gemini JSON	JSON parsing and validation errors are converted to readable <code>RuntimeErrors</code> .
Wrong data structure	The outline is checked to ensure it is a list of exactly five panels.
Gemini 504 timeout	The generation helper retries the request and reports failure if attempts are exhausted.
Gemini 503 unavailable	The helper retries; the final error identifies the temporary service condition.
Invalid character profile	Pydantic validation prevents malformed profile data from continuing silently.
HTTP generation failure	FastAPI returns an error response and logs the traceback for debugging.

Testing note

A 503 UNAVAILABLE response means the external Gemini service may be experiencing temporary high demand. It should be distinguished from application-code errors such as invalid parameters or malformed JSON.

19. Testing and Validation

Test area	What was verified	Observed result
Homepage	Application loads in browser	PASS
Gemini connectivity	Gemini models were discoverable from the configured client	PASS
Story generation	Five-panel outline generation worked during successful runs	PASS
Character consistency	Locked character profile was used across panel prompts	PASS
Image generation	Panel PNGs were generated and served successfully	PASS
Preview	Generated panel assets were displayed in the application	PASS
Speech bubbles	Preview dialogue presentation was tested	PASS / reviewed
PDF export	Successful Fox story reached final PDF output	PASS
Different story prompts	Multiple stories were attempted	Tested
External-service resilience	503/504 conditions were observed and logged	Handled / documented

The testing process deliberately used more than one story so that the system was not validated only against a single fixed prompt. Successful generations and temporary Gemini availability errors were both recorded as part of realistic testing.

20. Advantages

- Converts a simple story idea into a structured comic workflow.
- Reduces manual prompt-writing effort.
- Uses a reusable character profile for better identity consistency.
- Separates story planning from image prompting.
- Provides a dedicated comic preview rather than only raw generated images.
- Supports speech-bubble presentation.
- Produces a PDF suitable for demonstration and sharing.
- Built with modular Python services that can be extended later.

21. Limitations

- Generative-image models can still produce visual variation between panels.
- External Gemini availability can affect generation time or success.
- Long or repeated generation requests can take time.
- AI-generated content may require human review before final publication.
- PDF dialogue rendering and browser-preview rendering are separate presentation stages and should be tested independently.
- Current workflow is centered on a five-panel comic structure.

22. Future Enhancements

- Support variable panel counts instead of a fixed five-panel structure.
- Allow users to edit individual panel prompts before regeneration.

- Provide a reusable character library.
- Add multiple comic-page templates and layouts.
- Add more direct control over speech-bubble shape and placement.
- Support multilingual story and dialogue generation.
- Add voice-to-story or voice-to-comic input.
- Provide direct image and project-package downloads.
- Add user accounts and saved comic projects.
- Add additional AI image-generation providers or selectable models.
- Add mobile-friendly/PWA support.

23. Applications

- Educational storytelling and classroom activities.
- Children's creative writing and visual learning.
- Comic prototyping and storyboarding.
- Short-form entertainment content.
- Concept visualization for writers and creators.
- AI-assisted creative media demonstrations.
- Academic demonstrations of generative-AI integration.

24. Conclusion

ComicCraft demonstrates an end-to-end approach to AI-assisted comic creation. Starting from a basic prototype webpage, the project evolved into a more polished web application with a dedicated comic-generation workflow, structured Gemini integration, character consistency logic, generated panel artwork, dialogue presentation and PDF export.

The project also demonstrates an important engineering principle: generative AI should be combined with validation, application logic and presentation controls rather than treated as a single black-box step. By creating a structured character profile and reusing it in later prompts, ComicCraft improves the consistency of the generated story world.

The completed project provides a practical foundation for future enhancements such as editable panels, larger comic formats, multilingual support and reusable character libraries.

25. Demonstration Guide

The following sequence can be used during a live project demonstration.

22. Open the ComicCraft homepage.
23. Briefly explain how the interface evolved from the original basic prototype to the current polished design.
24. Enter a short story prompt and choose the character, setting, tone and art style.
25. Click Generate and explain that Gemini first creates the five-panel outline.
26. Explain the permanent character profile and why it is generated before image prompts.
27. Show the generated comic panels and point out character consistency.
28. Show the speech bubbles and explain that dialogue is handled separately from the artwork.
29. Open the export option and demonstrate PDF creation.
30. Open the PDF and show the final comic output.
31. Briefly mention validation and temporary external Gemini errors as part of realistic AI-service behavior.

32. Conclude with the future enhancements planned for ComicCraft.

Suggested closing line

“ComicCraft demonstrates how generative AI, structured backend services and a user-friendly interface can work together to transform a simple story idea into a complete comic experience.”

26. References

- Google Gemini API / Google AI for Developers documentation – model usage, generation and troubleshooting.
- FastAPI documentation – backend web framework and routing.
- Python documentation – core programming language reference.
- Pydantic documentation – data validation and model definitions.
- PyTorch documentation – tensor and machine-learning framework support.
- Hugging Face Diffusers documentation – diffusion-based image-generation pipelines.
- Project source code and configuration developed for ComicCraft AI.

Appendix A – Key Project Components

Component	Known responsibility
app/services/gemini_flash.py	Gemini client, text generation, five-panel outline, character profile and image-prompt generation.
app/routes.py	Application route handling and generation orchestration.
app/config.py	Application settings and Gemini model/API configuration.
app/models.py	Pydantic request/output structures including PromptRequest, CharacterProfile and PanelOutline.
comic_preview.html	Generated comic preview page with panel information and export action.
layout builder / layout logic	Combines panel data, dialogue and presentation information.
PDF exporter	Creates the final downloadable/printable PDF representation.
static/panels	Serves generated panel PNG assets to the browser.

Appendix B – Current Gemini Generation Logic

The current implementation creates a Gemini client using the configured API key and a five-minute HTTP timeout. The generation helper retries transient failures before reporting a final RuntimeError. The outline stage explicitly validates that Gemini returned exactly five panels.

```
raw_outline = _generate_text(
    client,
    settings.gemini_flash_model,
    outline_prompt,
)

# Character identity is generated separately
character_profile = _generate_character_profile(
    client,
    settings.gemini_flash_model,
    request,
)
```

THANK YOU

PROJECT COMPLETED BY

Vidya Sara.A.M. (Team Lead)

Varshini.S

Swathi.A

Tamilarasi

Viveka.p

GOVERNMENT ARTS COLLEGE FOR WOMEN,SALEM-08